

Snowflakes

Day 1 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“bags of uniqueness”*
- ▶ *“longest contiguous segment of unique snowflakes”*

Programming interpretation:

- ▶ A valid package is a contiguous window with no repeated snowflake ID.
- ▶ When a duplicate appears inside the current window, the left edge must move past the previous copy.
- ▶ The largest valid window length is the maximum package size.

Textbook connection: CP4 2.3 f. Hash Table (map), Harder

Core technique: Sliding window with a hash map

- ▶ We need the longest contiguous sequence with no repeated snowflake ID.
- ▶ The current window [left, right] is always duplicate-free.
- ▶ A dictionary records the most recent index of each snowflake.

- ▶ Contiguous + unique elements points to a sliding window.
- ▶ A hash table tells us whether a duplicate is inside the active window.
- ▶ Moving left only forward keeps the total work linear.

- ▶ Initialize $\text{left} = 0$, $\text{best} = 0$, and an empty `last_seen` dictionary.
- ▶ Scan each snowflake as the right endpoint.
- ▶ If this snowflake was last seen at or after `left`, move `left` past that position.
- ▶ Update `last_seen` for the current snowflake.
- ▶ Update `best` with $\text{right} - \text{left} + 1$.

- ▶ Duplicates before left are irrelevant because they are outside the current window.
- ▶ A duplicate inside the window forces the shortest safe shift of left.
- ▶ The window never contains two copies of the same ID after the update.

- ▶ The invariant is that the maintained window is always duplicate-free.
- ▶ When a duplicate appears, any valid window ending at right must start after the previous copy.
- ▶ Since every right endpoint is considered and best stores the largest valid window, the final answer is optimal.

Complexity: $O(N)$ expected time per test case and $O(N)$ memory.

- ▶ Use `sys.stdin.buffer.read` for fast input across many test cases.
- ▶ Store positions, not counts, because the exact previous index determines the new left boundary.

Source lines: snowflakes.py:27-37

```
import sys

def solve():
    data = list(map(int, sys.stdin.buffer.read().split()))
    pos = 0

    test_cases = data[pos]
    pos += 1

    answers = []
```

What this chunk does

- ▶ The solution reads all integers once for fast input.
- ▶ pos is a cursor into the flat token list.
- ▶ answers collects one result per test case before printing.

Source lines: snowflakes.py:39-45

```
for _ in range(test_cases):  
    n = data[pos]  
    pos += 1  
  
    last_seen = {}  
    left = 0  
    best = 0
```

What this chunk does

- ▶ Each test case starts with n snowflake IDs.
- ▶ last_seen maps an ID to its most recent index.
- ▶ left and best describe the current unique window and best answer.

Source lines: snowflakes.py:47-57

```
for right in range(n):
    snowflake = data[pos]
    pos += 1

    if snowflake in last_seen and last_seen[snowflake] >= left:
        left = last_seen[snowflake] + 1

    last_seen[snowflake] = right

    best = max(best, right - left + 1)
```

What this chunk does

- ▶ right scans each snowflake exactly once.
- ▶ If a duplicate is still inside the window, left jumps after its old position.
- ▶ Updating last_seen after the jump records the newest occurrence.

Source lines: snowflakes.py:59-64

```
    answers.append(str(best))  
  
    print("\n".join(answers))  
  
solve()
```

What this chunk does

- ▶ best is appended after the whole stream for one test case is processed.
- ▶ A single joined print avoids repeated output calls.
- ▶ solve() is invoked at the bottom to run the workflow.

Solution file: `practice_contests/day_1/snowflakes.py`

Problem: <https://open.kattis.com/problems/snowflakes>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.